Process Checkpointing in Linux and the Architectural Barriers in Windows

Ibtisam Ahmed
contact@samstack.site

***Abstract*— Application-level hibernation allows individual processes to be checkpointed and later restored without suspending the entire operating system. In Linux, this capability is implemented by Checkpoint/Restore in Userspace (CRIU), which serializes process memory, file descriptors, and CPU register state. However, an equivalent mechanism does not exist in the Windows ecosystem.**

This paper investigates the architectural reasons behind this disparity. Through analysis of Linux CRIU internals and experimental exploration of Windows process suspension and memory serialization techniques, we identify key barriers that prevent straightforward application-level checkpointing on Windows. These barriers include the tight coupling between user processes and kernel-managed objects, such as handles, GDI resources, and security tokens.

Experimental prototypes demonstrate that while process execution can be temporarily suspended and private memory can be serialized, the restoration of a consistent execution environment remains an unresolved challenge in Windows.**Keywords—ACPI S4, CRIU, Windows OS, Process Hibernation, State Serialization.*

I. Introduction

We are all familiar with the modern convenience of closing a laptop lid and returning hours later to find every window exactly where we left it. This everyday magic is formally known as the ACPI S4 state, a deep hibernation that preserves the entire executed software context without requiring continuous power to your RAM. However, this global entire-memory approach brings significant latency and creates introduces potential security risks. To achieve those lightning-fast resume times, the operating system dumps a raw, unencrypted copy of your entire system memory directly to your permanent storage.

This raises a seemingly simple question: if we can hibernate an entire computer, why can't we hibernate just a single, resource-heavy application?

Modern applications are tightly coupled to the operating system through shared libraries, network sockets, and inter-process communication. Freezing a single app while the system continues running breaks these connections, leading to stale handles and invalid session references when the app resumes.

This work investigates the architectural differences between Linux and Windows that affect application-level checkpointing. Using CRIU as a reference implementation, we analyze the mechanisms required to capture and restore process state and experimentally explore which of these mechanisms can be replicated within the Windows process model.

II. BACKGROUND

A. The ACPI S4 State

In the Advanced Configuration and Power Interface (ACPI) specification, hibernation (a.k.a S4 state sleep) represents the deepest sleep state where the executed software context is preserved without requiring continuous power to the RAM. Unlike the S3 sleep state, S4 utilizes Operating System-directed configuration and Power Management (OSPM) to facilitate a total power-down. This process begins when the kernel invokes a freeze subsystem to halt all user-space and kernel threads, creating a memory image. To ensure data integrity, the system utilizes a copy-on-write (or copy while writing) mechanism to generate a snapshot of active memory pages, ensuring that the hibernation process itself does not mutate the state it is attempting to save.

The persistence of this state is managed by engines like swsusp (for Linux), which write a unique signature to the swap partition or a dedicated hibernation file, such as hiberfil.sys (in Windows). During the data transfer, the kernel's block I/O layer performs a dump from physical RAM to permanent storage. Modern implementations frequently prioritize LZO or LZ4 compression to minimize disk bandwidth bottlenecks, aiming for a resume-to-desktop time of under five seconds. However, this optimization often results in a security vacuum, as the OS performs a raw, unencrypted dump of system memory to the disk to maintain high-speed transitions, potentially exposing sensitive volatile data in a persistent format.

B. Linux Hibernation Architecture

The Linux kernel manages the transition to the ACPI S4 state primarily through the Software Suspend (swsusp) framework. This process is a rigorous exercise in state serialization, where the kernel must freeze the entire execution environment to ensure a small memory image. By invoking the freeze subsystem, the kernel halts all user-space processes and specific kernel threads, preventing data mutation during the snapshot creation. This snapshot is then mapped to the system's swap space,

either a dedicated partition or a swap file (depending on user's preference during installation), where the kernel writes the memory image alongside a specific signature in the swap header.

This signature serves as a critical flag for the bootloader and kernel during the POST, signaling that the system should bypass a standard cold boot in favor of a resume operation from the swap image.

On modern distributions, the orchestration of this transition is abstracted via systemd-sleep. This service acts as the high-level manager for power state changes, facilitating the execution of sleep hooks (scripts that run automatically before and after the system enters or exists the S4 sleep state) located in /usr/lib/systemd/system-sleep/. These hooks allow

for the injection of custom logic at two specific points: the pre phase, occurring before the kernel-level freeze, and the post phase, occurring immediately after the system resumes. This modularity allows admins to address the inherent security to latency trade off by unmounting encrypted volumes (such as LUKS or VeraCrypt) or flushing sensitive memory buffers before the raw RAM dump is committed to non-volatile storage.

C. Windows Hibernation Architecture

In the Windows ecosystem, hibernation is architected through the Power Manager and the kernel-mode power policy manager, which orchestrate the transition to the ACPI S4 state. Unlike the Linux swsusp framework that typically targets a swap partition, Windows utilizes a dedicated, hidden system file known as hiberfil.sys located at the root of the boot volume. When the hibernation signal is triggered, the kernel directs the Memory Manager to save the contents of RAM to this file. To optimize for the "Fast Resume" objective, Windows does not save every page, it typically filters out zeroed pages and caches, compressing the remaining active memory set before the disk write occurs.

The Windows boot process is uniquely modified by this state through a feature known as Hiberboot (or Fast Startup). In this mode, when a user selects Shut Down, Windows actually performs a partial hibernation by closing user sessions but saving the kernel session and loaded drivers to hiberfil.sys. Upon the next power-on, the Windows boot loader detects the

hibernation signature in the file header and reads the stored kernel image back into RAM. This bypasses the traditional, hardware-heavy enumeration of drivers and services, drastically reducing boot latency at the cost of leaving a persistent snapshot of the kernel state on the physical disk.

From a security architecture perspective, Windows hibernation presents a significant forensic vector. Unless BitLocker full-disk encryption is active, the hiberfil.sys file remains an unencrypted, raw dump of the system's volatile memory. This file can be parsed by forensic tools to recover encryption keys, user passwords, and fragments of decrypted documents that were present in RAM at the moment of the S4 transition. While Windows provides shutdown scripts via Group Policy, they lack the low-level granularity of Linux system-sleep hooks, often making it difficult to selectively purge sensitive memory buffers without significantly delaying the hibernation sequence.

* *"the modern hibernation file almost loses its value once the system is resumed, as all the file's content is zeroed, except the header."*

The core architectural discrepancy between full-system hibernation and per app hibernation lies in state and resource dependency. While ACPI S4 hibernation captures a static, global snapshot of the entire physical memory address space, individual applications do not exist in a vacuum. A modern, resource-heavy application like a browser is deeply entwined with the host Operating System through shared dynamic link libraries (DLLs or .so files), open network sockets, file descriptors, and inter-process communication (IPC) handles. To hibernate a single app, the OS would need to serialize not just the application's private memory, but also the volatile state of every kernel-level hook and hardware-abstracted resource it is currently utilizing.

The primary technical hurdle is the resuming to context consistency. When a full system resumes from S4, the entire world, including the kernel and all hardware drivers, is restored to the exact millisecond it was frozen.

In contrast, if an individual application were restored into a live OS that has continued to evolve, the application's external references would be broken. Network sockets would have timed out at the protocol level, and memory addresses for shared system resources might have been reallocated to other processes. This creates a stale handle problem where the restored application attempts to communicate with a kernel environment that no longer recognizes its session IDs or security tokens.

III. EXPERIMENTAL INVESTIGATION

A. The Linux Approach: CRIU

The Checkpoint/Restore In Userspace (CRIU) project represents the most robust Linux implementation of per app hibernation, operating primarily through the manipulation of the /proc filesystem and the ptrace() (in C/C++) system call. Unlike the kernel-level swsusp which operates at the hardware abstraction layer, CRIU functions in user-space to perform process surgery. The tool begins by seizing control of the target process tree using ptrace(PTRACE_SEIZE), which halts execution and allows CRIU to inject a parasite code into the process's address space. This parasite acts as a local agent to dump the CPU registers, thread contexts, and memory maps into a series of image files, effectively externalizing the internal state of the process.

A critical barrier to seamless application restoration is File System and Resource Consistency. Since CRIU does not snapshot the entire disk, it is highly sensitive to the stale handle problem. To mitigate this, CRIU utilizes "Ghost Files", temporary storage for the contents of deleted or modified files that were open during the checkpoint, to ensure that the restored process still sees its expected data at the correct offsets. However, if a file path changes or an external pipe is broken, the restoration typically fails. This necessitates a rigid adherence to the Mount Namespace. For a successful resume, the process must be re-inserted into an environment where every file descriptor and memory-mapped file matches the pre-hibernation state exactly.

The technical contribution of CRIU lies in how it sews the process back into the kernel tree upon restore. This is a multi-stage re-animation process: first, CRIU creates a restorer process that uses the clone() system call with the CLONE_NEWPID flag to recreate the original Process ID (PID) hierarchy. Once the PID tree is reconstructed, the restorer maps the dumped memory pages back into their original virtual addresses and uses the prctl() system call to restore the process's metadata, such as its name and resource limits. Finally, it jumps into the restored instruction pointer (IP) and executes sigreturn, a technique that tricks the kernel into restoring the entire CPU register state from a fake signal frame, effectively stitching the process back into the scheduler as if it had never been interrupted.

To demonstrate process checkpointing, a simple Bash script was created that continuously increments a counter once per second. During execution, the process is checkpointed using CRIU, which dumps its memory state to disk. The process is then restored, allowing execution to resume from the previously saved state.

```
Listing 1: Sample counter script (counter.sh)

1. #!/bin/bash
2. i=0
3. while true; do
4.   echo "Counter: $i"
5.   i=$((i+1))
6.   sleep 1
7. done
```

Figure 1: Process checkpoint and restoration using CRIU

```
(a) Initial execution of the counter script
testlinux@testlinux:~/Desktop$ ./counter.sh
Counter: 0
Counter: 1
Counter: 2
Counter: 3
Counter: 4
Counter: 5
Counter: 6
Counter: 7
Counter: 8
Counter: 9
Counter: 10
Counter: 11
Counter: 12
Counter: 13
Counter: 14
Counter: 15
Counter: 16
Counter: 17
Counter: 18
Counter: 19
Counter: 20
Counter: 21
Killed

(b) Process checkpoint using CRIU
testlinux@testlinux:~/Desktop$ pgrep -f counter.sh
4786
testlinux@testlinux:~/Desktop$ sudo criu dump -t 4786 -D ./dump_data --shell-job

(c) Process restoration and continued execution
testlinux@testlinux:~/Desktop$ sudo criu restore -D ./dump_data --shell-job
Counter: 22
Counter: 23
Counter: 24
Counter: 25
Counter: 26
```

As shown in Figure 1(a), the script begins counting from zero during normal execution.

In Figure 1(b), the running process is checkpointed using CRIU, which serializes the process state to disk.

After restoration (Figure 1(c)), execution resumes from counter value 22, demonstrating that the process state was successfully preserved and restored.

The most significant distinction here is that upon calling criu restore, the process does not simply restart, it resumes execution from the exact state of its last increment (Counter 22), effectively tricking the CPU into believing the process was never terminated.

This success is rooted in the Linux kernel's native support for process introspection. While the Windows implementation relies on manually incrementing a SuspendCount for individual threads via kernel32.dll to keep them in RAM, CRIU performs parasitic injection to externalize the process's entire address space, registers, and file descriptors to the disk. On Windows, the header acts as an external supervisor keeping the app's kernel resource dependency alive in active memory.

B. The Windows Way

The primary obstacle in transposing Linux-style hibernation to the Windows environment is the architectural entanglement of the Windows subsystem. While Linux adheres to a philosophy of discrete, modular resources often represented as files, Windows applications are fundamentally bound to the Windows Executive and the Object Manager. In this ecosystem, a high-level application is rarely a standalone entity. It is supported by an overarching network, shared GDI (Graphics Device Interface) handles, and dynamic dependencies on services like svchost.exe or lsass.exe, tight coupling with kernel-managed resources. If one attempts to hibernate a Windows app by merely dumping its private memory, the restoration will fail because the application's hooks into the OS, its window handles, hardware-accelerated buffers, and security tokens, would have been invalidated by the live kernel in the meantime.

To explore whether CRIU-like behavior could be replicated in Windows, a prototype supervisory process was developed, that acts as a specialized manager that tracks these dependencies by monitoring the process tree and its associated system calls. Unlike Linux, which can halt a process with a single SIGSTOP, Windows requires a more granular intervention via SuspendThread in kernel32.dll. Since Windows has no concept of a process-level pause, the header must iterate through the process's thread pool and increment the Suspend Count for every individual execution thread. This forces the Windows Scheduler to ignore these threads, effectively freezing the application in physical RAM. This state represents a suspended animation where the memory remains intact, but the execution logic is dormant.

My proof of concept is by using psutil to demonstrate the first phase of this per app hibernation: the Isolation of Temporal State (status of the program at a certain point in time). By successfully pausing the counter and resuming it without a loss of sequence, I will have proven that the application's internal register state and heap remain consistent within a live RAM environment. However, to evolve this into true hibernation, the header must perform a Memory-to-Disk Serialization. This involves dumping the private virtual address space of the suspended process into a file.

i. Stage I - Temporal Isolation

To demonstrate temporal isolation, a supervisory header process was implemented to control the lifecycle of a target application. The header process can launch, suspend, and resume the application using operating system process control primitives.

```
Listing 2: Header process responsible for
controlling the target application.

1. import subprocess
2. import psutil
3.
4. def run_manager():
5.     target_process = None
6.
7.     while True:
8.         print("\nProcess Control")
```

```
 9.         print("1. Launch App")
10.         print("2. Suspend")
11.         print("3. Resume")12.         print("4.
Kill and Exit")
13.
14.         choice = input("Select: ")
15.
16.         if choice == '1':
17.             target_process =
subprocess.Popen(["python","myapp.py"])
18.             print(f"App started with id:
{target_process.pid}")
19.
20.         elif choice == '2':
21.             if target_process:
22.
psutil.Process(target_process.pid).suspend()
23.
24.         elif choice == '3':
25.             if target_process:
26
psutil.Process(target_process.pid).resume()
```

```
Listing 3: Target application used to demonstrate
temporal isolation.

1. import time
2.
3. i = 0
4. while True:
5.     print(f"Counting: {i} at time:
{time.strftime('%H:%M:%S')}")
6.     i += 1
7.     time.sleep(1)
```

Figure 2: Suspension and resumption of the target process

```
PS C:\Users> python header.py

Process Control
1. Launch App
2. Suspend
3. RESUME
4. Kill and Exit
Select: 1
App started with id: 4840

Counting: 0 at time: 22:50:54
Counting: 1 at time: 22:50:55
Counting: 2 at time: 22:50:56
Counting: 3 at time: 22:50:57
Counting: 4 at time: 22:50:58
Counting: 5 at time: 22:50:59
Select: 2
PID 4840 is now frozen.

Process Control
1. Launch App
2. Suspend
3. RESUME
4. Kill and Exit
Select: 3
PID 4840 resumed.
Counting: 6 at time: 22:51:03
Counting: 7 at time: 22:51:04
Counting: 8 at time: 22:51:05
Counting: 9 at time: 22:51:06
Counting: 10 at time: 22:51:07
Select: 4
```

This demonstrates temporal state isolation via supervised thread suspension, establishing the first stage required for granular application hibernation.

ii. Stage 2 - Memory Serialization

To achieve true application-level hibernation, the volatile state must be entirely serialized to non-volatile disk storage. This ensures the application state survives system reboots or the termination of the controlling Header program.

This demonstrates the successful extraction of the target application's private address space, heap data, and CPU thread context, utilizing OS-level mechanisms analogous to VirtualQueryEx and MiniDumpWriteDump. The serialization process is verified through three distinct forensic proofs.

Before a process can be serialized, the managing Header must map the application's address space to identify which regions contain vital state data (such as the stack and private heap) versus static mapped files (such as shared OS libraries).

```
Listing 4: Enumerating the process memory map using
psutil.

 1. import os
 2. import psutil
 3.
 4. magic_marker = b"MAGIC_STRING" * 50000
 5.
 6. pid = os.getpid()
 7. print(f"PID: {pid}")
 8.
 9. print(f"{'Address Range':<35} | {'Size
(RSS)':<10} | {'Path/Type'}")
10. p = psutil.Process(pid)
11. maps = p.memory_maps(grouped=False)
12.
13. for m in maps[:15]:
```

```
14.     path_display = m.path if m.path else
"[Anonymous / Heap / Stack]"
15.     print(f"{m.addr:<35} | {m.rss:<10} |
{path_display}")
16.
17. input("\n")
```

Figure 3: Enumeration of the process memory regions, showing address ranges, resident set size (RSS), and mapped file paths.

```
PS C:\Users\     \OneDrive\Desktop> python mem.py
PID: 23504
Address Range                       | Size (RSS) | Path/Type
0x1e9ff6a0000                       | 69632      | C:\Windows\System32\C_1252.NLS
0x1e9ff6c0000                       | 69632      | C:\Windows\System32\C_437.NLS
0x1e9ff6e0000                       | 12288      | C:\Windows\System32\l_intl.nls
0x1e9ff720000                       | 12288      | C:\Windows\System32\l_intl.nls
0x1e9ff730000                       | 864256     | C:\Windows\System32\locale.nls
0x1e9ff810000                       | 69632      | C:\Windows\System32\C_1252.NLS
0x1e9ff830000                       | 69632      | C:\Windows\System32\C_437.NLS
0x1ea02920000                       | 65536      | C:\Program Files\WindowsApps\python3.dll
0x7ff7111f0000                      | 4096       | C:\Program Files\WindowsApps\python3.13.exe
```

Application memory map enumeration, detailing Resident Set Size (RSS), memory region addresses, and file paths. This mapping dictates the boundaries for the subsequent memory dump.

To prove that volatile, dynamically allocated memory is successfully captured during the dump process, a unique marker string (MAGIC_STRING) was injected into the target application's private heap prior to suspension. Once the application was suspended and serialized to a .DMP file, a forensic binary analysis was conducted on the resulting file.

The analysis successfully located the exact byte sequence residing at a static hex offset within the dump file, proving that the active state of the application's memory was perfectly preserved on the disk.

Figure 4: Forensic verification of serialized memory. The injected marker string MAGIC_STRING is located within the memory dump, confirming that dynamically allocated heap data was preserved during serialization.

```
python.3.14.DMP
Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  Decoded text

00E59D20  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 54  ...............T
00E59D30  01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00E59D40  C0 7C 02 EA 01 00 00 10 5F 1E DB FF 7F 00 00 01  À|.ê...._.Ûÿ....
00E59D50  00 00 00 02 00 00 00 F0 3F 00 00 D0 3F 00 00 01  .......ð?..Ð?...
00E59D60  00 00 00 00 00 00 00 20 74 19 DB FF 7F 00 00 0F  ....... t.Ûÿ....
00E59D70  00 00 00 00 00 00 00 FF FF FF FF FF FF FF FF 62  .......ÿÿÿÿÿÿÿÿb
00E59D80  22 4D 41 47 49 43 5F 53 54 52 49 4E 47 22 00 03  "MAGIC_STRING"..
00E59D90  00 00 00 00 00 00 00 20 74 19 DB FF 7F 00 00 0C  ....... t.Ûÿ....
00E59DA0  00 00 00 00 00 00 00 3A 05 2B 15 64 CA AD 6C 4D  .......:.+.dÊ.lM
00E59DB0  41 47 49 43 5F 53 54 52 49 4E 47 00 00 00 00 01  AGIC_STRING.....
00E59DC0  00 00 00 00 00 00 00 20 74 19 DB FF 7F 00 00 05  ....... t.Ûÿ....
00E59DD0  00 00 00 00 00 00 00 FF FF FF FF FF FF FF FF 35  .......ÿÿÿÿÿÿÿÿ5
00E59DE0  30 30 30 30 00 00 00 00 00 00 00 00 00 00 00 01  0000............
00E59DF0  00 00 00 00 00 00 00 20 74 19 DB FF 7F 00 00 03  ....... t.Ûÿ....
00E59E00  00 00 00 00 00 00 00 FF FF FF FF FF FF FF FF 70  .......ÿÿÿÿÿÿÿÿp
00E59E10  69 64 00 00 00 00 00 00 00 00 00 00 00 00 00 01  id..............
00E59E20  00 00 00 00 00 00 00 20 74 19 DB FF 7F 00 00 02  ....... t.Ûÿ....
```

Saving the application's data is only half of the hibernation equation, to resume execution, the CPU must know the exact instruction it was processing at the moment of suspension.

This requires capturing the thread context, most notably the Instruction Pointer (RIP) and the Stack Pointer (RSP).

Analysis of the generated memory dump confirms that the precise architectural state of the CPU threads was serialized alongside the memory pages.

Figure 5: CPU thread context extracted from the dump, highlighting the instruction pointer (RIP) and stack pointer (RSP), which determine the exact execution location of the suspended thread.

```
Command
0:000> r
rax=0000000000000007 rbx=0000000000000064 rcx=0000000000000064
rdx=0000000000000000 rsi=0000006ff05eeac0 rdi=0000000000000004
rip=00007ff892961b34 rsp=0000006ff05ee908 rbp=0000000001000005
 r8=0000000000000000  r9=0000000000000000 r10=0000000000000064
r11=0000000000000064 r12=00007fffdb251be0 r13=0000000000000064
r14=0000000000000000 r15=0000006ff05eec80
iopl=0         nv up ei pl zr na po nc
cs=0033  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00000246
ntdll!NtDeviceIoControlFile+0x14:
00007ff8`92961b34 c3              ret
0:000> lm
start             end                 module name
000001ea`02920000 000001ea`02930000   python3    (export symbols)       python3.dll
00007ff7`111f0000 00007ff7`1121b000   python3_13   (no symbols)
00007ff8`6bed0000 00007ff8`6bee6000   _psutil_windows   (deferred)
00007ff8`6c050000 00007ff8`6c066000   _socket    (deferred)
00007ff8`6c4a0000 00007ff8`6c4be000   VCRUNTIME140   (deferred)
00007ff8`6e4f0000 00007ff8`6e557000   Windows_Storage_ApplicationData   (deferred)
00007ff8`71800000 00007ff8`71809000   select     (deferred)
00007ff8`75f40000 00007ff8`75f96000   pdh        (deferred)
00007ff8`7fb60000 00007ff8`7fb7a000   windows_staterepositorycore   (deferred)
00007ff8`84540000 00007ff8`84784000   twinapi_appcore   (deferred)
00007ff8`89330000 00007ff8`8933b000   version    (deferred)
```

This Header Process model effectively shifts the responsibility of state management from the OS kernel to a specialized supervisor.

By tracking the kernel resource dependencies, the specific Service IDs and Handle IDs an app uses, the header can theoretically re-request these resources from the OS during the resume phase. While this adds overhead to the hibernation transition, it circumvents the natural nature of the ACPI S4 state. In a Windows context, this is an adequate middle ground: it provides the persistence of hibernation with the granularity of process management, assuming the header can successfully spoof or re-initialize the application's connection to the Windows Executive.

While Windows lacks a native 'Restore' command for process dumps, we demonstrate that the state is successfully serialized. A third-party Header can theoretically achieve persistence by utilizing Process Hollowing techniques to re-inject this serialized state into a fresh process shell, bridging the gap between volatile suspension and non-volatile hibernation

IV. SECURITY CONCERNS

While the Header process model addresses the latency and granularity issues of ACPI S4, it introduces unique Security and Integrity Risks. By externalizing a single application's memory to a file, the system creates a new forensic target. Without integrating encryption into the Memory-to-Disk serialization phase, sensitive application data, such as browser session tokens or decrypted document fragments, becomes persistently stored on the disk, mirroring the vulnerabilities of hiberfil.sys but at a more targeted level. Furthermore, the lack of a global freeze during per app hibernation means that concurrent system processes could potentially modify shared files or resources before the restoration is complete, leading to data corruption or stale handle crashes.

V. LIMITATIONS

Despite the theoretical viability of the proposed Header-based architecture for per app hibernation on Windows, several structural limitations within the Windows operating system prevent reliable state restoration. These limitations arise from deep kernel-user space dependencies that cannot be faithfully reconstructed from user-mode memory snapshots alone.

A. Kernel Object Dependencies

Windows processes maintain extensive relationships with kernel-managed objects such as events, file objects, sockets, and device handles. These objects reside in kernel memory and are referenced through opaque handles stored in the process handle table. When a process is suspended and its memory serialized, the underlying kernel objects themselves are not captured. Upon restoration, the revived process may attempt to reference kernel objects whose state has changed or whose underlying kernel structures no longer exist. Because these objects are managed by the Windows Object Manager within kernel space, user-mode restoration mechanisms cannot deterministically recreate them.

B. Address Space Layout Randomization (ASLR)

Modern Windows systems enforce Address Space Layout Randomization (ASLR) as a core security mitigation. ASLR causes executable images, libraries, heaps, and stacks to be loaded at randomized virtual addresses during process initialization. The Header architecture requires deterministic reconstruction of the original virtual address layout in order to correctly restore serialized pointers, thread stacks, and register contexts. However, ASLR interferes with this requirement by preventing predictable address placement. While it is theoretically possible to modify Portable Executable headers to disable ASLR for a target process, this approach introduces compatibility concerns and weakens system security guarantees. Consequently, reliable restoration of the exact memory layout remains difficult without bypassing or disabling this mitigation.

C. Handle Table Reconstruction

Each Windows process maintains a handle table that maps user-mode handle values to kernel object references. This table is dynamically managed by the Windows kernel and contains internal metadata that is not directly accessible from user space. During restoration, simply recreating numeric handle values is insufficient because the underlying kernel references may point to objects with different states or lifetimes. Without reconstructing the exact handle table entries maintained by the kernel, restored applications may encounter invalid handle errors, broken synchronization primitives, or unexpected resource states. This represents a fundamental barrier to accurate process resurrection in Windows without kernel-level cooperation.

D. IPC State Desynchronization

Inter-process communication (IPC) mechanisms such as shared memory segments, named pipes, and sockets introduce additional consistency challenges. If shared memory regions are serialized as part of the process snapshot, they may diverge from the state maintained by peer processes that continued executing during the suspension period. This can lead to severe state desynchronization when the application resumes. The proposed Hybrid Header attempts to mitigate this issue by treating IPC resources as external state: prior to suspension the application is instructed to detach from shared memory regions, and upon restoration it re-establishes IPC channels. While this strategy reduces corruption risks, it does not fully eliminate inconsistencies when peer processes maintain persistent state that cannot be easily renegotiated.

VI. CONCLUSION

The evolution of hibernation from a global system state to a granular, application-specific process reveals a fundamental architectural conflict between convenience and security. While the standard ACPI S4 state provides a reliable method for total power-down, its full-memory nature creates a security vacuum by committing raw, unencrypted RAM dumps to persistent storage. The umbilical cord problem, the deep entanglement of software with kernel-mode objects, remains the primary barrier to per app hibernation, particularly in the Windows ecosystem.

This investigation demonstrates that while Linux provides native primitives required for application-level checkpointing, the Windows process architecture presents several structural barriers that prevent a direct equivalent. The investigated Header process model successfully achieves Isolation of Temporal State by manually incrementing thread-level Suspend Counts through kernel32.dll. However, true per app hibernation on Windows remains contingent on the manager's ability to re-stitch severed handles during restoration. Ultimately, the transition to granular hibernation shifts the responsibility of state management from the OS kernel to specialized supervisors, offering a revolutionary middle ground for modern power and resource management.

REFERENCES

The template will number citations consecutively within brackets [1]. The sentence punctuation follows the bracket [2]. Refer simply to the reference number, as in [3]—do not use “Ref. [3]” or “reference [3]” except at the beginning of a sentence: “Reference [3] was the first ...”

Number footnotes separately in superscripts. Place the actual footnote at the bottom of the column in which it was cited. Do not put footnotes in the abstract or reference list. Use letters for table footnotes.

Unless there are six authors or more give all authors’ names; do not use “et al.”. Papers that have not been published, even if they have been submitted for publication, should be cited as “unpublished” [4]. Papers that have been accepted for publication should be cited as “in press” [5]. Capitalize only the first word in a paper title, except for proper nouns and element symbols.

For papers published in translation journals, please give the English citation first, followed by the original foreign-language citation [6].

[1] Microsoft, "System Sleeping States," Microsoft Learn, [Online]. Available: https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/system-sleeping-states/.

[2] S. M. M. R. Khalil, "Analysis of Windows 10 Hibernation File,"

[3] Linux Programmer's Manual, "systemd-sleep(8) Linux manual page." [Online].

[4] P. Emelyanov, "Checkpoint/restore mostly in the userspace," Linux Kernel Mailing List, Jul. 2011. [Online]. Available: https://lore.kernel.org/all/4E204466.8010204@parallels.com/.

[5] CRIU, "CRIU:About - CRIU," criu.org, Dec. 26, 2013. [Online]. Available: https://criu.org/CRIU:About.

[6] G. Rodola, "psutil documentation," readthedocs.io,. [Online]. Available: https://psutil.readthedocs.io/.

[7] Microsoft, "SuspendThread function (processthreadsapi.h) - Win32 apps," Microsoft Learn, Feb. 22, 2024. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-suspendthread.

[8] Linux Programmer's Manual, "ptrace(2) - Linux manual page," man7.org, Jan. 16, 2026. [Online]. Available: https://man7.org/linux/man-pages/man2/ptrace.2.html.

[9] Linux Programmer's Manual, "clone(2) - Linux manual page," man7.org. [Online]. Available: https://man7.org/linux/man-pages/man2/clone.2.html.

[10] Linux Programmer's Manual, "prctl(2) - Linux manual page," man7.org, Jan. 16, 2026. [Online]. Available: https://man7.org/linux/man-pages/man2/prctl.2.html.

[11] MITRE, "Process Injection: Process Hollowing, Sub-technique T1055.012," MITRE ATT&CK, Oct. 24, 2025. [Online]. Available: https://attack.mitre.org/techniques/T1055/012/.

[12] Linux Programmer's Manual, "sigreturn(2) - Linux manual page," man7.org. [Online].